Database Systems ACE6283

# *Student Club Event Manager*

## Phase 1&2

Group Members:

| Student Name | Student ID |
|---|---|
| Yakoub Murad Abdulqawi | 1221305670 |
| Rian Salem Banawair | 1231300718 |
| | |
| | |

# *Student Club Event Manager*

**Database Systems Group Project (ACE6283)**
**Phase 1 Report**

---

## 1. Executive Summary

**System Name:** ClubFlow (Student Club Event Management System)

**Objective:** The primary objective of this project is to **design and implement a normalized relational database schema for "ClubFlow."** This database will serve as the centralized back-end for managing student club operations, specifically focusing on event coordination, member engagement, and attendance verification.

**Outcome:** The final database design will be normalized to the **Third Normal Form (3NF)** to eliminate data redundancy and ensure transactional integrity. By implementing rigorous Primary/Foreign Key relationships and Check Constraints, the system will provide a "single source of truth" for club administrators.

**Key Deliverables:**

- **User Interfaces (UI/UX)**

  Interactive Figma prototypes showcasing the system's front-end design.

  Interfaces for Member Registration, Event Management, Attendance Tracking, and Admin Dashboards.

- **Conceptual Design:** Entity Relationship Diagram (ERD) detailing entities like Members, Events, and Registrations.

- **Logical Design:** A relational schema mapping out 3NF compliance.

- **Physical Implementation:** SQL DDL scripts for table creation and constraint enforcement.

- **Data Manipulation:** SQL DML scripts for generating attendance and registration reports.

---

## 2. Requirements Analysis and Scope Definition

### 2.1 Business Domain and Purpose

**Current Business Problem (As-Is):** Student clubs currently rely on fragmented, manual processes—primarily messaging apps (WhatsApp/Telegram) and paper-based sign-in sheets. This decentralized approach has led to:

> **Data Inconsistency:** Discrepancies between registration lists and actual attendance records.
>
> **Administrative Overhead:** Manual consolidation of member data from various platforms for every new event.
>
> **Poor Reporting:** Difficulty in identifying active vs. inactive members or calculating event success metrics for university funding reports.

**Proposed Solution (To-Be):** The proposed database will power a centralized platform that streamlines the event lifecycle. It will allow organizers to publish events, enable members to register via a digital profile, and provide admins with real-time visibility into attendance trends.

## 2.2 Detailed Functional Requirements

**FR-01:** The system shall allow members to register and log in using their student ID and email address to ensure unique identification of each user.

**FR-02:** The system shall allow organizers to create and manage events by providing event details such as title, date, time, venue, and description.

**FR-03:** The system shall allow members to view a list of upcoming events and register for events of interest.

**FR-04:** The system shall prevent duplicate registrations by ensuring that a member cannot register for the same event more than once.

**FR-05:** The system shall allow organizers to record attendance by marking registered members as present or absent during or after an event.

**FR-06:** The system shall allow administrators to generate reports showing event registrations and attendance statistics.

**FR-07:** The system shall send confirmation notifications to members after successful event registration.

**FR-08:** The system shall allow members to cancel their event registration before the event date, with the registration status updated accordingly.

---

## 2.3 List of Core Entities

Based on the requirements analysis, the system will be built around the following five core entities:

**1-Members** (MemberID, Name, Email, StudentID) – stores student info

2-**Events** (EventID, OrganizerID, Title, Date, Time, Venue, Description) – stores event details

3-**Registrations** (RegID, MemberID, EventID, Status) – links members to events

4-**Attendance** (AttendanceID, MemberID, EventID, PresentAbsent) – tracks event participation

5-**Organizers** (UserID, Role, Name, Email_org) – stores admin/organizer info

---

## 2.4 Key Business Rules & Constraints

To ensure data integrity, the following rules will be implemented at the database level:

**Entity Integrity (Uniqueness):**

Every **Member** must have a unique `StudentID`.

Every **Event** must have a unique `EventID`.

**Referential Integrity (Valid References):**

A **Registration** cannot exist without a valid `MemberID` and `EventID`.

Deleting an **Event** should perform a "Cascade Delete" or "Restrict" on associated registrations to prevent orphaned records.

**Implementation:** Foreign Key (FK) constraints on the `Registration` table.

**Domain Integrity (Data Validity):**

**Mandatory Fields:** `Date` and `Name` are set to `NOT NULL`.

**Logical Consistency:** An Event's Date must be greater than or equal to the current system date (no past events can be created).

**Default Values:** Registration status defaults to `'Registered'` until an organizer updates it to `'Attended'`.

---

## 2.5 Non-Functional Requirements (NFR)

**NFR-01:** The system shall be accessible via a mobile application on both Android and iOS platforms.

**NFR-02:** The system shall respond to user interactions and load application screens within two seconds under normal operating conditions.

**NFR-03:** The system shall support at least 200 concurrent active users without performance degradation.

**NFR-04:** All communication between the client application and the server shall be secured using HTTPS.

**NFR-05:** The user interface shall be optimized for mobile devices, providing touch-friendly controls and intuitive navigation.

**NFR-06:** The system shall perform daily data backups to ensure data availability and recovery in case of failure.

---

# 3. (Interfaces)

The following screens will be designed using **Figma**:

**Login / Register (Page 1)**

Student ID

Email

**Event List (Page 2)**

Displays upcoming events

**Register in the Event( Page 3 )**

Event description

Name, ID

major

**Organizer Dashboard(page 4, page 5, page 6, page 7, page 8)**

Create events

View registrations

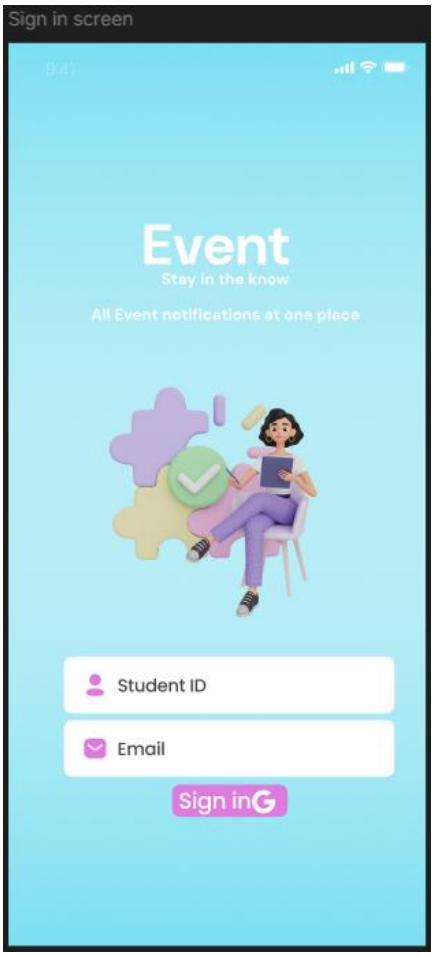
Mark attendance

View attendance reports

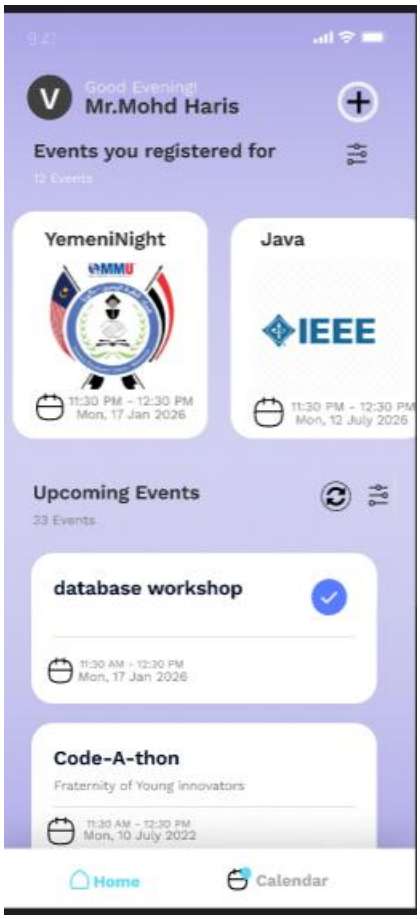
View registration statistics

Manage members

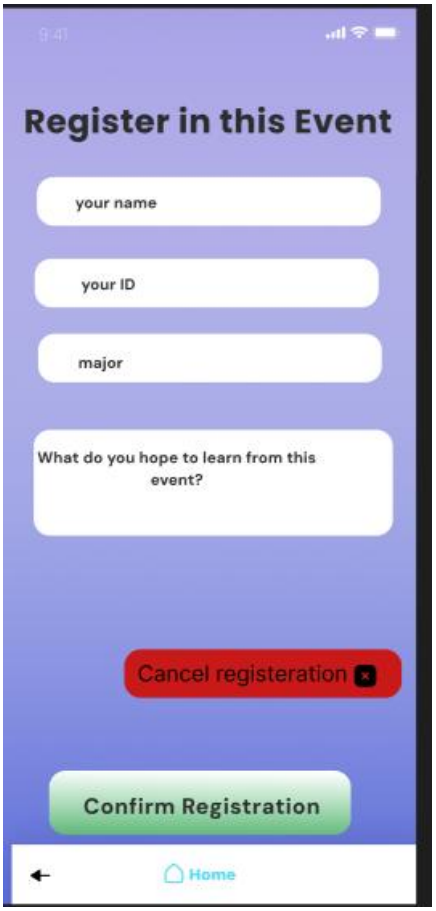
Page 1



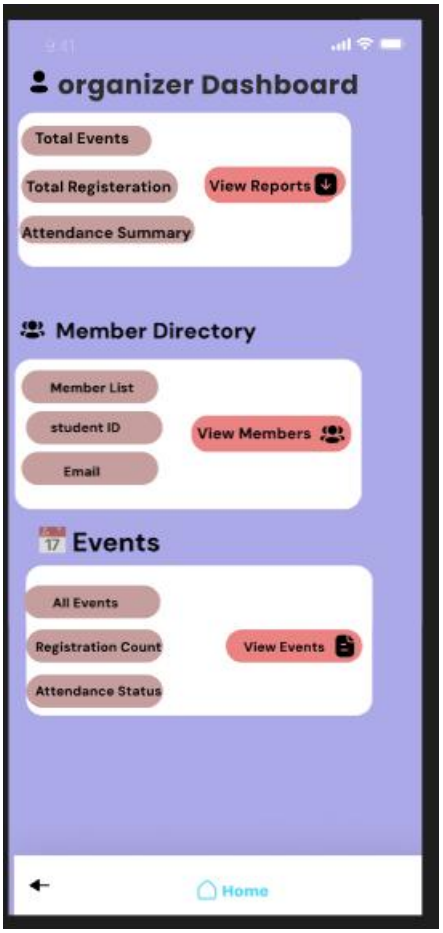

page 2




Page 3



Page 4

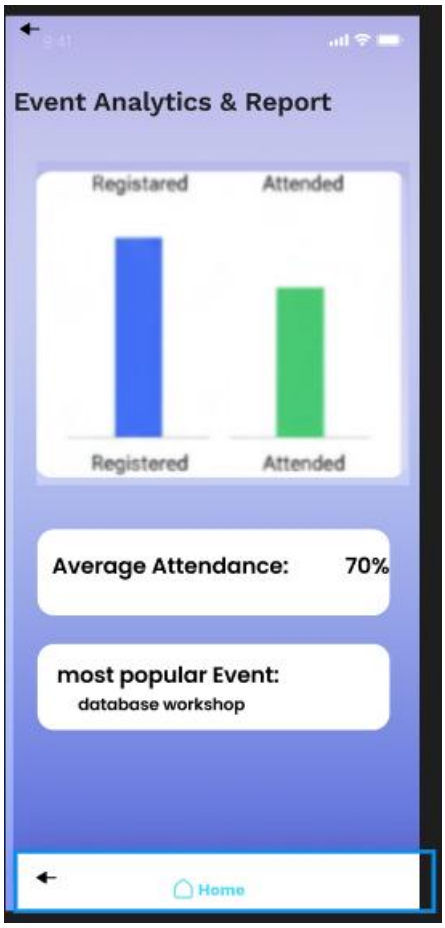



page 5

page 6

Page 7

Page 8



# 4. Conceptual Design

This section translates the functional requirements of **ClubFlow** into a high-level, system-independent structural model. It defines the entities, their key attributes, and the cardinalities that govern how data interacts within the system.

## 4.1 Entity and Attribute Discovery

The following entities have been identified to support the business rules defined in Section 2. Each entity is designed to ensure data integrity while minimizing redundancy.

**1. Entity: Members**

**Description:** Represents the students who interact with the club by registering for and attending events.

**Attributes:**

`MemberID` (Internal Unique Identifier / Primary Key)

`StudentID` (University-issued ID; unique constraint)

`Name` (Full name of the student)

`Email` (Official university email used for communication and login)

**2. Entity: Events**

**Description:** Represents a specific activity or gathering organized by the club.

**Attributes:**

`EventID` (Unique Identifier)

`OrganizerID` (Foreign Key referencing Organizers created the event)

`Title` (The name of the event)

`Date` (The scheduled calendar day

`Time` (The specific start time)

`Venue` (Location of the event)

`Description` (Details regarding the event's purpose

**3. Entity: Registrations (Associative Entity)**

**Description:** A transactional entity that records a member's intent to attend a specific event. This resolves the Many-to-Many relationship between Members and Events.

**Attributes:**

`RegID` (Unique Transaction Identifier)

`MemberID` (Foreign Key referencing Member)

`EventID` (Foreign Key referencing Event)

`Status` (Registration status: 'Registered', 'Cancelled')

**4. Entity: Attendance**

**Description:** Records the physical presence of a member at an event. While linked to Registration, it is separated to allow for walk-in tracking or distinct verification by organizers.

**Attributes:**

`AttendanceID` (Unique Identifier)

`MemberID` (Foreign Key referencing Member)

`EventID` (Foreign Key referencing Event)

`PresentAbsent` (Attendance status: 'Present' or 'Absent')

**5. Entity: Organizers**

**Description:** Represents the club committee members who have administrative privileges to manage events.

**Attributes:**

`UserID` (Unique Identifier)

`Name` (Full name of the organizer)

`Email_org` (Contact and login credential)

`Role` (e.g., President, Secretary, Event Coordinator)

---

## 4.2 Relationship Analysis

The following relationships define how data entities interact within the **ClubFlow** system. We utilize standard cardinality notation to ensure the database enforces correct business logic.

**. Members to Registrations (1:N)**

**Relationship:** A Member signs up for events.

**Cardinality:** One Member can have **Zero or Many** Registrations over their tenure. A specific Registration record belongs to exactly One Member.

**Rule:** `Member (1) ||--o{ Registration (N)`

**2. Events to Registrations (1:N)**

**Relationship:** An Event receives sign-ups.

**Cardinality:** One Event can have **Zero or Many** Registrations. A specific Registration record is associated with exactly One Event.

**Rule:** `Event (1) ||--o{ Registration (N)`

**3. Organizers to Events (1:N)**

**Relationship:** An Organizer/Admin creates and manages events.

**Cardinality:** One Organizer can create **One or Many** Events. Each Event is managed by at least One Organizer.

**Rule:** `Organizer (1) ||--|{ Event (N)`

**4. Members to Attendance (1:N)**

**Relationship:** A Member's physical presence is recorded.

**Cardinality:** One Member can have **Zero or Many** Attendance records (one for each event they physically attend). Each Attendance record identifies exactly One Member.

**Rule:** `Member (1) ||--o{ Attendance (N)`

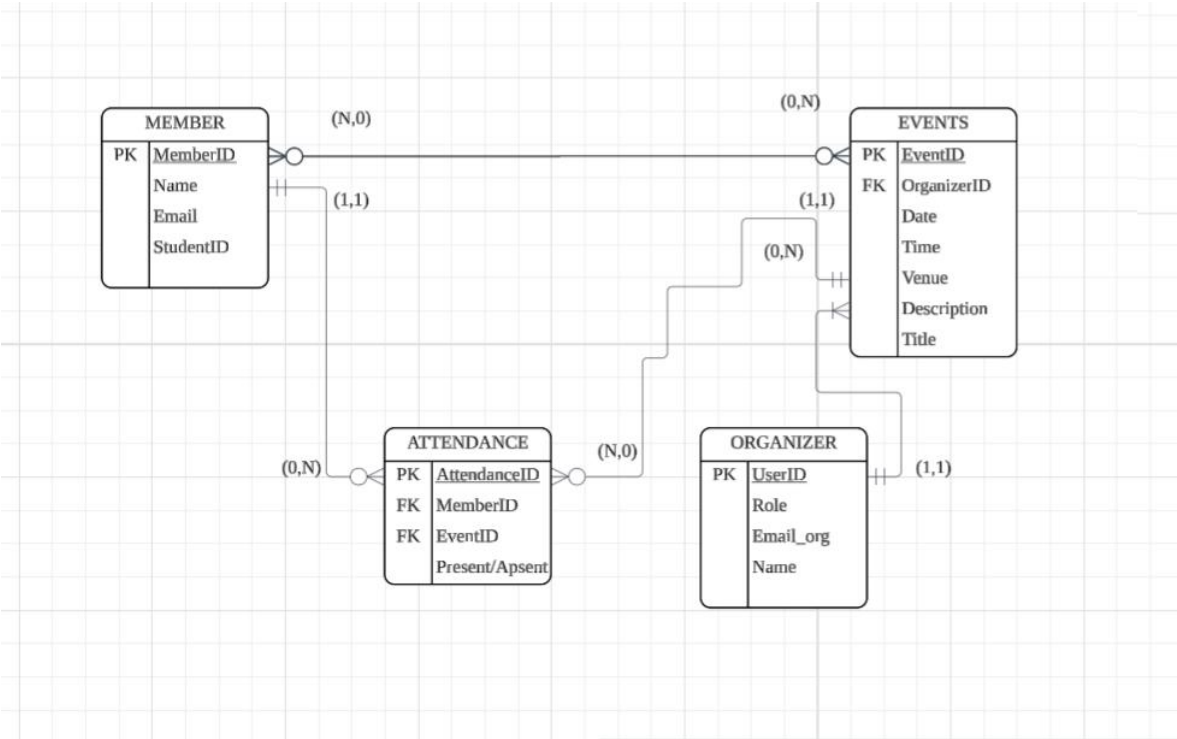
**5. Events to Attendance** PK **(1:N)**

**Relationship:** An Event tracks who showed up.

**Cardinality:** One Event can have **Zero or Many** Attendance marks. A specific Attendance record refers to exactly One Event.

**Rule:** `Event (1) ||--o{ Attendance (N)`

3.3 Conceptual ER Diagram :The diagram below illustrates the conceptual structure of the club database.




Note on Many-to-Many Relationships:

To resolve the conceptual Many-to-Many (M:N) relationship between Members and Events (as one member attends many events and one event hosts many members), the system utilizes Registration and Attendance as associative entities. This structure eliminates redundancy and ensures the database maintains Third Normal Form (3NF) compliance.

## 5. Logical Design and Normalization

This section details the conversion of the conceptual model into a logical database structure, defining specific data types and relationships.

### 5.1 Entity and Attribute Definitions (Data Dictionary)

The following data dictionary defines the structure for each table, including data types and constraints.

## Entity: Members

| Attribute | Data Type | Description |
|---|---|---|
| MemberID | INT (PK, Auto Increment) | Unique identifier for each club member. |
| Name | VARCHAR(100) | Full name of the member. |
| Email | VARCHAR(100), UNIQUE | Member's email address used for login and notifications. |
| StudentID | VARCHAR(20) | Official student ID assigned by the university. |

## Entity: Events

| Attribute | Data Type | Description |
|---|---|---|
| EventID | INT (PK, Auto Increment) | Unique identifier for each event. |
| OrganizerID | INT (FK → Organizers.UserID) | The organizer who creates the event |
| Title | VARCHAR(150) | Name/title of the event. |
| Date | DATE | Scheduled date of the event. |
| Time | TIME | Scheduled time of the event. |
| Venue | VARCHAR(100) | Location where the event will be held. |
| Description | TEXT | Detailed description of the event. |

## Entity: Registrations

| Attribute | Data Type | Description |
|---|---|---|
| RegID | INT (PK, Auto Increment) | Unique identifier for each registration record. |
| MemberID | INT (FK → Members.MemberID) | Links registration to the member who registered. |
| EventID | INT (FK → Events.EventID) | Links registration to the event. |
| Status | VARCHAR(20) | Current status of registration (e.g., Registered, Cancelled). |

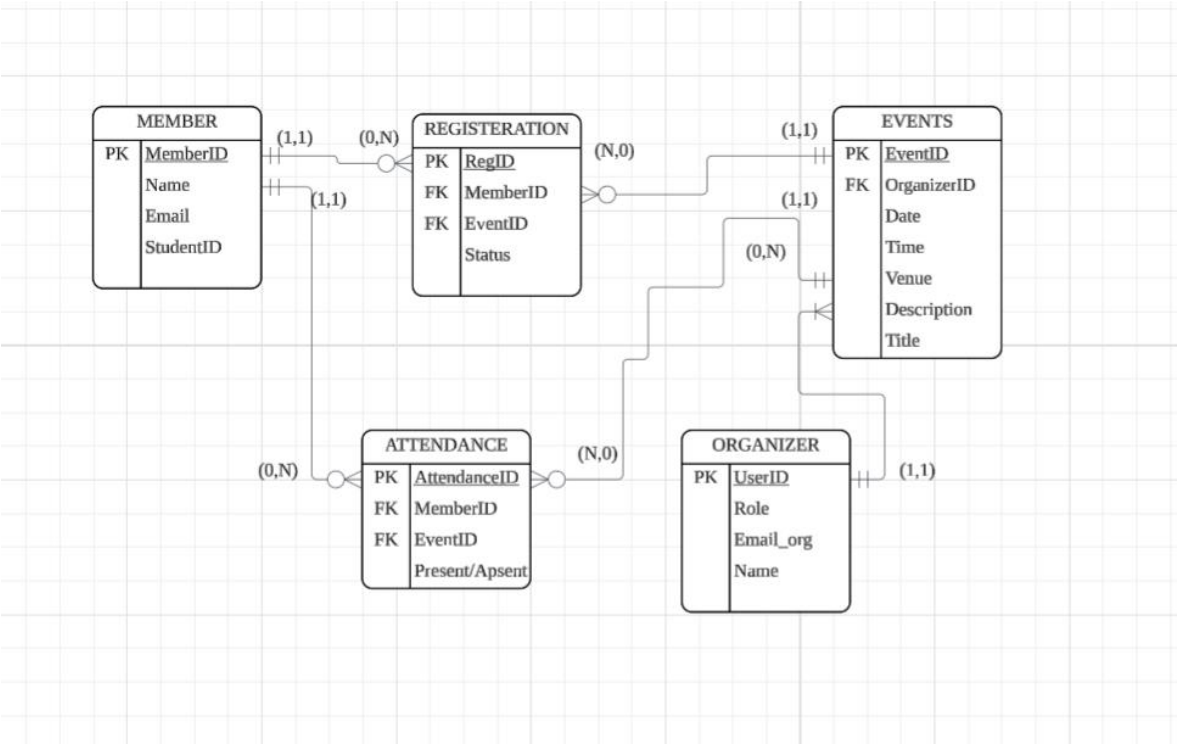
## Entity: Attendance

| Attribute | Data Type | Description |
|---|---|---|
| AttendanceID | INT (PK, Auto Increment) | Unique identifier for each attendance record. |
| MemberID | INT (FK → Members.MemberID) | Links attendance to the member. |
| EventID | INT (FK → Events.EventID) | Links attendance to the event. |
| PresentAbsent | VARCHAR(10) | Attendance status: 'Present' or 'Absent'. |

## Entity: Organizers

| Attribute | Data Type | Description |
|---|---|---|
| UserID | INT (PK, Auto Increment) | Unique identifier for each organizer/admin. |
| Role | VARCHAR(20) | Defines role (Organizer, Admin). |
| Name | VARCHAR(100) | Full name of the organizer/admin. |
| Email_org | VARCHAR(100), UNIQUE | Email address used for login and communication. |

## 5.2 Relational Schema Crow's Foot ERD (Logical)




**Text Schema**

**Members** (MemberID (pk), Name, Email (unique), StudentID)

**Events** (EventID (pk), OrganizerID (fk → Organizers.UserID), Title, Date, Time, Venue, Description)

**Registrations (**RegID (pk), MemberID (fk → Members.MemberID), EventID (fk → Events.EventID), Status)

**Attendance (**AttendanceID (pk), MemberID (fk → Members.MemberID), EventID (fk → Events.EventID), PresentAbsent)

**Organizers** (UserID (pk), Role, Name, Email_org (unique)

# Relationship Explanation (Crow's Foot Logic)

### 1. Members ↔ Registrations

**Type:** One-to-Many

One member can have many registrations

One registration belongs to exactly one member

Member participation is optional

---

## 2. Events ↔ Registrations

type: One-to-Many

One event can have many registrations, One registration belongs to exactly one event, Event participation is optional

➡ Together, **Member** ↔ **Event** is **Many-to-Many**, resolved by **Registration**

---

## 3. Members ↔ Attendance

**Type:** One-to-Many

One member can have many attendance records, Attendance must belong to a member

---

## 4. Events ↔ Attendance

**Type:** One-to-Many

One event can have many attendance records, Attendance must belong to an event

➡ Together, **Member** ↔ **Event** is **Many-to-Many**, resolved by **Attendance**

---

## 5. Organizers ↔ Events

**Type:** One-to-Many

One organizer can manage many events, Each event is created by one organizer

---

# *5.3 Database Tables Normalization*

**Justification for Third Normal Form (3NF):**

**1. First Normal Form (1NF):**

All tables have a defined Primary Key (MemberID, EventID, RegID, AttendanceID, UserID).

All attributes contain atomic values. For example, MemberName is stored as a single unit, and Event details such as Title, Date, Time, and Venue are stored separately rather than combined.

No repeating groups exist; each record represents one entity instance (e.g., one member, one event, one registration).

**2. Second Normal Form (2NF):**

All tables are already in 1NF.

There are no partial dependencies because all Primary Keys are single attributes (surrogate keys).

Example: In the Registration table, the Status attribute depends only on RegID. Since RegID is the sole primary key, no non-key attribute depends on part of a composite key.

**3. Third Normal Form (3NF):**

All tables are in 2NF.

There are no transitive dependencies, meaning non-key attributes depend only on the primary key and not on other non-key attributes.

Example: In the Event table, Title, Date, Time, Venue, and Description depend only on EventID. In the Organizer/Admin table, Role, Name, and Email depend only on UserID. Attributes such as OrganizerName are not stored in the Event table directly; instead, UserID is used as a Foreign Key referencing the Organizer/Admin table. This ensures proper separation of data and compliance with 3NF.

**Conclusion:** The Member, Event, Registration, Attendance, and Organizer/Admin tables are all normalized up to 3NF. This design eliminates redundancy, prevents anomalies, and ensures consistency, supporting the system's goal of accurate event management for the

# 6. Physical Design

This section contains the implementation details. The Logical Design is translated into specific SQL (Structured Query Language) scripts compatible with standard relational database management systems (e.g., MySQL, PostgreSQL, SQL Server).

## 6.1 Testing and Compliance

To ensure the database meets the requirements defined in Section 2, the following testing strategy was employed:

1-**Constraint Testing:** Attempted to insert a Registration for a MemberID that does not exist.

**Expected Result:** Database error (Foreign Key Constraint Violation).

2-**Uniqueness Testing:** Attempted to register two members with the same Email.

**Expected Result:** Database error (Unique Constraint Violation).

**3-Reporting:** Executed complex queries to join Members, Events, Registrations, and Attendance to verify relationships are functioning correctly.

## 6.2 Appendix: SQL Scripts

```
-- ==========================================
-- A. DDL: Create Tables
-- ==========================================

-- 1. Members Table
CREATE TABLE Members (
   MemberID INT PRIMARY KEY AUTO_INCREMENT,
   Name VARCHAR(100) NOT NULL,
   Email VARCHAR(100) NOT NULL UNIQUE,
   StudentID VARCHAR(20) NOT NULL UNIQUE
);

-- 2. Organizers Table
CREATE TABLE Organizers (
   UserID INT PRIMARY KEY AUTO_INCREMENT,
   Name VARCHAR(100) NOT NULL,
   Role VARCHAR(20) NOT NULL,
   Email_org VARCHAR(100) NOT NULL UNIQUE
);

-- 3. Events Table
CREATE TABLE Events (
```

```
    EventID INT PRIMARY KEY AUTO_INCREMENT,
    Title VARCHAR(150) NOT NULL,
    Date DATE NOT NULL,
    Time TIME NOT NULL,
    Venue VARCHAR(100) NOT NULL,
    Description TEXT,
    OrganizerID INT NOT NULL,
    FOREIGN KEY (OrganizerID) REFERENCES Organizers(UserID) ON DELETE RESTRICT
);

-- 4. Registrations Table
CREATE TABLE Registrations (
    RegID INT PRIMARY KEY AUTO_INCREMENT,
    MemberID INT NOT NULL,
    EventID INT NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Registered',
    UNIQUE (MemberID, EventID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID) ON DELETE CASCADE,
    FOREIGN KEY (EventID) REFERENCES Events(EventID) ON DELETE CASCADE,
    CHECK (Status IN ('Registered', 'Cancelled'))
);

-- 5. Attendance Table
CREATE TABLE Attendance (
    AttendanceID INT PRIMARY KEY AUTO_INCREMENT,
    MemberID INT NOT NULL,
    EventID INT NOT NULL,
    PresentAbsent VARCHAR(10) NOT NULL,
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID) ON DELETE CASCADE,
    FOREIGN KEY (EventID) REFERENCES Events(EventID) ON DELETE CASCADE,
    CHECK (PresentAbsent IN ('Present', 'Absent'))
);


-- B. DML: Insert Sample Data
-- ==========================================

-- Organizers
```

```
INSERT INTO Organizers (Name, Role, Email_org)
VALUES
('Ali Ahmed', 'President', 'ali@club.edu'),
('Sara Khaled', 'Secretary', 'sara@club.edu');

-- Members
INSERT INTO Members (Name, Email, StudentID)
VALUES
('Ahmed Saleh', 'ahmed@student.edu', 'S1001'),
('Fatima Noor', 'fatima@student.edu', 'S1002'),
('Omar Hassan', 'omar@student.edu', 'S1003'),
('Aisha Mohammed', 'aisha@student.edu', 'S1004');

-- Events
INSERT INTO Events (Title, Date, Time, Venue, Description, OrganizerID)
VALUES
('Annual Cultural Night', '2026-03-01', '18:00:00', 'FOM Empty Space', 'Cultural gathering and Iftar',
1),
('Tech Talk 2026', '2026-04-10', '14:00:00', 'Lecture Hall A', 'Technology and innovation talk', 2);

-- Registrations
INSERT INTO Registrations (MemberID, EventID, Status)
VALUES
(1, 1, 'Registered'),
(2, 1, 'Registered'),
(3, 1, 'Cancelled'),
(4, 1, 'Registered'),
(1, 2, 'Registered'),
(2, 2, 'Registered');

-- Attendance
INSERT INTO Attendance (MemberID, EventID, PresentAbsent)
VALUES
(1, 1, 'Present'),
(2, 1, 'Absent'),
(4, 1, 'Present'),
(1, 2, 'Present'),
```

```
(2, 2, 'Present');

-- C. Sample Queries
-- ==========================================

-- Query 1: List all members registered for a specific event (FR-03)
-- Purpose: Allows organizers to view the registration list for a particular event
-- to manage logistics and expected turnout.
SELECT Members.Name, Events.Title, Registrations.Status
FROM Registrations
JOIN Members ON Registrations.MemberID = Members.MemberID
JOIN Events ON Registrations.EventID = Events.EventID
WHERE Events.Title = 'Annual Cultural Night';

-- Query 2: Show attendance status for a specific event (FR-05)
-- Purpose: Retrieves the actual attendance records to verify which registered
-- students physically attended the event.
SELECT Members.Name, Events.Title, Attendance.PresentAbsent
FROM Attendance
JOIN Members ON Attendance.MemberID = Members.MemberID
JOIN Events ON Attendance.EventID = Events.EventID
WHERE Events.Title = 'Annual Cultural Night';

-- Query 3: Generate a report of registrations and attendance for admin (FR-06)
-- Purpose: Provides a summary report comparing total registrations vs. actual
-- attendance for every event.
SELECT
   Events.Title,
   COUNT(DISTINCT Registrations.RegID) AS TotalRegistrations,
   COUNT(DISTINCT Attendance.AttendanceID) AS TotalAttendance
FROM Events
LEFT JOIN Registrations ON Events.EventID = Registrations.EventID
LEFT JOIN Attendance ON Events.EventID = Attendance.EventID
   AND Attendance.PresentAbsent = 'Present'
GROUP BY Events.EventID, Events.Title;
```

```sql
-- Query 4: List all events created by a specific organizer
-- Purpose: Allows administrators to view all events managed by a particular
-- organizer for accountability and workload tracking.
SELECT
    Organizers.Name AS OrganizerName,
    Organizers.Role,
    Events.Title AS EventTitle,
    Events.Date AS EventDate,
    Events.Time AS EventTime,
    Events.Venue,
    Events.Description
FROM Events
JOIN Organizers ON Events.OrganizerID = Organizers.UserID
WHERE Organizers.Name = 'Ali Ahmed'
ORDER BY Events.Date DESC;
```

## -- Business Query Questions

```sql
-- Query 1: Members registered for "Annual Cultural Night"
SELECT
    Members.Name,
    Members.Email
FROM Registrations
JOIN Members ON Registrations.MemberID = Members.MemberID
JOIN Events ON Registrations.EventID = Events.EventID
WHERE Events.Title = 'Annual Cultural Night';

-- Query 2: Complete event list with organizer details
SELECT
    Events.Title AS EventTitle,
    Events.Date AS EventDate,
    Events.Venue,
    Organizers.Name AS OrganizerName,
    Organizers.Role AS OrganizerRole
FROM Events
JOIN Organizers ON Events.OrganizerID = Organizers.UserID
ORDER BY Events.Date;
```

```sql
-- Query 3: Total "Present" count per event
SELECT
   Events.Title AS EventTitle,
   COUNT(Attendance.AttendanceID) AS TotalPresent
FROM Events
LEFT JOIN Attendance ON Events.EventID = Attendance.EventID
   AND Attendance.PresentAbsent = 'Present'
GROUP BY Events.EventID, Events.Title
ORDER BY Events.Title;

-- Query 4: Tech Talk registrants who didn't attend or were absent
SELECT
   Members.Name,
   Members.StudentID
FROM Registrations
JOIN Members ON Registrations.MemberID = Members.MemberID
JOIN Events ON Registrations.EventID = Events.EventID
LEFT JOIN Attendance ON Registrations.MemberID = Attendance.MemberID
   AND Registrations.EventID = Attendance.EventID
   AND Attendance.PresentAbsent = 'Present'
WHERE Events.Title = 'Tech Talk 2026'
   AND Registrations.Status = 'Registered'
   AND Attendance.AttendanceID IS NULL;

-- Query 5: Event report with attendance rate
SELECT
   Events.Title AS EventTitle,
   COUNT(DISTINCT CASE
      WHEN Registrations.Status = 'Registered'
      THEN Registrations.RegID
   END) AS TotalRegistered,
   COUNT(DISTINCT CASE
      WHEN Attendance.PresentAbsent = 'Present'
      THEN Attendance.AttendanceID
   END) AS TotalPresent,
   ROUND(
```

```
        COUNT(DISTINCT CASE WHEN Attendance.PresentAbsent = 'Present'
            THEN Attendance.AttendanceID END) * 100.0 /
        NULLIF(COUNT(DISTINCT CASE WHEN Registrations.Status = 'Registered'
             THEN Registrations.RegID END), 0),
        2
    ) AS AttendanceRate
FROM Events
LEFT JOIN Registrations ON Events.EventID = Registrations.EventID
LEFT JOIN Attendance ON Events.EventID = Attendance.EventID
GROUP BY Events.EventID, Events.Title
ORDER BY Events.Title;
```

Youtube link

(92) Database systems presentation - YouTube